ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных технологий
Кафедра Информатики и информационных технологий

направление подготовки
09.04.02 «Информационные системы и технологии»

Лабораторная работа №4

Дисциплина: Искусственный интеллект в мобильных системах.

Тема: Разработка программы распознания и генерации речи.

Выполнил: студент группы 244-331
Гусев Леонид Васильевич
(Фамилия И.О.)

Дата, подпись: 09.04.2026 _____
(Дата) (Подпись)

Проверил: _____ _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись: _____ _____
(Дата) (Подпись)

Замечания:

_____

_____

_____

Москва

2026

ОГЛАВЛЕНИЕ

Введение 3
Задача 4
Практическая часть 5
Вывод 7

Введение

В ходе выполнения практического занятия была разработана программа голосового ассистента на языке Python. Основная цель работы заключалась в изучении методов распознавания и синтеза речи с использованием библиотек SpeechRecognition, gTTS и PyAudio, а также в реализации набора голосовых команд с озвучиванием ответов.

Программный код - https://github.com/nialegu/ai_ms_labs/tree/main/4

Задача

Написать программу на Python, которая бы воспринимала 7-8 голосовых команд и реагировала на голосовую команду stop - окончить работу. Ответы программа должна выдавать в виде голосовых сообщений.

Практическая часть

Программная система реализована в одном файле speech_recognition.py и состоит из нескольких логических модулей, отвечающих за распознавание речи, синтез ответа и обработку пользовательских команд.

Синтез речи

Озвучивание ответов реализовано в функции speak(). В качестве генератора речи используется библиотека gTTS, которая преобразует текст в аудиофайл формата MP3. Далее воспроизведение осуществляется с помощью модуля pygame.mixer. После завершения проигрывания временный файл автоматически удаляется. Параллельно текст ответа выводится в консоль.

Распознавание речи

Распознавание голосовых команд реализовано в функции listen_command(). Для захвата аудиопотока используется библиотека SpeechRecognition и микрофон через PyAudio. Перед записью выполняется автоматическая калибровка фонового шума. Распознавание текста осуществляется через Google Speech Recognition API с указанием русского языка (ru-RU). В случае ошибки распознавания или отсутствия интернет-соединения пользователь получает голосовое уведомление.

Работа с датой и временем

Функции get_time_text() и get_date_text() преобразуют системное время и дату в удобочитаемую русскоязычную форму. При этом учитываются правила склонения часов и минут, а также корректные названия месяцев.

Обработка команд

Логика обработки голосовых команд реализована в функции handle_command(). Поддерживаются следующие команды:

1. «Стоп», «Выход» — завершение работы программы.

2. «Сколько времени», «Который час» — озвучивание текущего времени.

3. «Какая дата», «Какое сегодня число» — вывод текущей даты.

4. «Открой почту» — открытие сайта mail.ru в браузере.

5. «Открой YouTube» — запуск видеохостинга YouTube.

6. «Кто ты», «Как тебя зовут» — представление ассистента.

7. «Погода» — вывод заглушки погодного сообщения.

8. «Анекдот» — воспроизведение шутки.

При вводе неизвестной команды ассистент сообщает, что не понял запрос, и просит повторить его.

Основной цикл работы

После запуска программа приветствует пользователя и переходит в бесконечный цикл ожидания команд. После каждой обработки ассистент уточняет, требуется ли дополнительная помощь. Завершение работы происходит при произнесении команд «стоп» или «выход».

```
(venv) leonidgusev@MacBook-Pro-Leonid 4 % python sr.py
[Распознано]: запусти youtube
[Ассистент]: Запускаю YouTube.
[Ассистент]: Есть ли ещё задачи для выполнения?

[Ожидание команды...]
[Ассистент]: Я не расслышала команду, попробуйте ещё раз.

[Ожидание команды...]
[Ассистент]: Я не расслышала команду, попробуйте ещё раз.

[Ожидание команды...]
[Распознано]: сколько времени
[Ассистент]: Сейчас 18 часов 23 минуты
[Ассистент]: Есть ли ещё задачи для выполнения?

[Ожидание команды...]
[Ассистент]: Я не расслышала команду, попробуйте ещё раз.

[Ожидание команды...]
[Распознано]: погода
[Ассистент]: Погода пока не подключена. Лучше посмотреть в окно или открыть прогноз.
[Ассистент]: Есть ли ещё задачи для выполнения?

[Ожидание команды...]
[Распознано]: анекдот
[Ассистент]: Почему программисты путают Хэллоуин и Рождество? Потому что Oct 31 == Dec 25!
[Ассистент]: Есть ли ещё задачи для выполнения?

[Ожидание команды...]
[Распознано]: стоп
[Ассистент]: Завершаю работу. Хорошего дня!
```

Вывод

В процессе выполнения лабораторной работы был разработан полнофункциональный Telegram-бот на языке Python, реализующий диалоговую систему с использованием методов NLP и машинного обучения. Бот использует трёхуровневую модель формирования ответов: ML-классификатор намерений, поиск подходящего ответа в датасете диалогов, а также фразы-заглушки при отсутствии релевантного совпадения. Рекламный модуль обеспечивает ненавязчивую интеграцию рекламы трёх товаров по четырём независимым сценариям, при этом исключая чрезмерно частую повторяемость сообщений.

В ходе работы были освоены методы классификации намерений, применение расстояния Левенштейна для нечеткого сопоставления строк, а также интеграция Python-приложения с Telegram Bot API.